

Coex User-Defined Function Kinds: Implementation Specification

Overview

User-defined function kinds provide lightweight syntax for embedding domain-specific languages (DSLs) in Coex. The compiler does not parse or understand the DSL body—it captures it as raw text and passes it to a handler function that implements the actual behavior.

This enables library authors to create first-class syntax for SQL, GraphQL, shell commands, LLM prompts, configuration formats, and other declarative languages without requiring compiler changes.

Design Goals

1. **Syntactic convenience:** DSLs look native, not like string literals passed to functions
 2. **Swappable implementations:** Same kind name, different backends (PostgreSQL vs MySQL)
 3. **Zero compiler knowledge:** Compiler treats body as opaque text
 4. **Type safety at boundaries:** Parameters and return types are checked
 5. **Configuration from environment:** Credentials and connections never in code
-

Syntax

Kind Declaration

A library declares a function kind with the `kind` keyword:

```
coex  
  
kind NAME -> RETURN_TYPE via HANDLER_FUNCTION
```

Example:

```
coex  
  
kind sqlquery -> QueryResult via sql_handler
```

This declares:

- `sqlquery` as a function kind available to users
- Functions of this kind return `QueryResult`
- The `sql_handler` function processes calls

Kind Usage

Users write functions using the declared kind:

```
coex

sqlquery get_users(min_age: int, active: bool) -> QueryResult:
    SELECT * FROM users
    WHERE age > $1
    AND active = $2
~
```

The body syntax is entirely determined by the library. The compiler captures it as-is.

Parameter Handling

The compiler does NOT perform any substitution. It packages:

- Parameter names (in declaration order)
- Parameter values (at call time)
- Raw body text (untouched)

And passes everything to the handler. The handler decides how to use parameters.

Different libraries use different conventions:

```
coex
```

```
# PostgreSQL: positional placeholders
sqlquery get_user(id: int) -> QueryResult:
    SELECT * FROM users WHERE id = $1
~

# Template library: curly braces
template greet(name: string, age: int) -> string:
    Hello {{name}}, you are {{age}} years old.
~

# Shell library: percent signs
shell list_files(dir: string) -> ShellResult:
    find %dir% -type f
~

# LLM library: just reference by position or ignore
llm summarize(text: string, bullets: int) -> Response:
    Summarize the following in {bullets} bullet points:

    {text}
~
```

Each library documents its convention. The compiler is agnostic.

Desugaring

The compiler transforms kind functions into regular function calls.

Input

```
coex

sqlquery get_active_users(min_age: int, dept: string) -> QueryResult:
    SELECT name, email FROM users
    WHERE age > $1
    AND dept = $2
    AND active = true
~
```

Called As

```
coex
```

```
result = get_active_users(25, "Engineering")
```

Desugared Output

```
coex

func get_active_users(min_age: int, dept: string) -> QueryResult:
  return sql_handler(KindCall(
    name: "get_active_users",
    param_names: ["min_age", "dept"],
    param_values: [min_age, dept],
    body: "SELECT name, email FROM users\nWHERE age > $1\nAND dept = $2\nAND active = true"
  ))
~
```

The body is passed exactly as written. The handler interprets `$1` and `$2` however it wants.

The KindCall Type

```
coex

type KindCall:
  name: string      # Function name
  param_names: [string] # Parameter names in declaration order
  param_values: [any]  # Parameter values at call time
  body: string       # Raw body text, untouched
~
```

Handler Functions

A handler function processes `KindCall` and returns the declared type. The handler is responsible for interpreting the body and parameters however it sees fit.

Signature

```
coex

func HANDLER_NAME(call: KindCall) -> RETURN_TYPE
```

Example: SQL Handler (Parameterized Queries)

```
coex
```

```

func sql_handler(call: KindCall) -> QueryResult:
    conn = get_connection_from_env("DATABASE_URL")

    # Library convention: $1, $2, etc. are positional parameters
    # Handler builds a prepared statement for safety
    stmt = conn.prepare(call.body)
    return stmt.execute(call.param_values)

~

```

Example: Template Handler (Named Placeholders)

```

coex

func template_handler(call: KindCall) -> string:
    result = call.body

    # Library convention: {{name}} placeholders
    for i in range(len(call.param_names)):
        placeholder = "{{" + call.param_names[i] + "}}"
        result = replace(result, placeholder, to_string(call.param_values[i]))
    ~
    return result
~

```

Example: Shell Handler (Percent Placeholders)

```

coex

func shell_handler(call: KindCall) -> ShellResult:
    command = call.body

    # Library convention: %name% placeholders with shell escaping
    for i in range(len(call.param_names)):
        placeholder = "%" + call.param_names[i] + "%"
        escaped_value = shell_escape(to_string(call.param_values[i]))
        command = replace(command, placeholder, escaped_value)
    ~

    return exec_command(command)
~

```

Example: LLM Handler (Curly Brace Placeholders)

coex

```
func llm_handler(call: KindCall) -> LLMResponse:
```

```
    api_key = get_env("ANTHROPIC_API_KEY")
```

```
    prompt = call.body
```

```
    # Library convention: {name} placeholders
```

```
    for i in range(len(call.param_names)):
```

```
        placeholder = "{" + call.param_names[i] + "}"
```

```
        prompt = replace(prompt, placeholder, to_string(call.param_values[i]))
```

```
    ~
```

```
    return anthropic_complete(api_key, prompt)
```

```
    ~
```

Library Structure

A complete library providing a function kind includes:

coex

```

# postgresql.coex

# External C functions for database access
extern pg_connect(connection_string: string) -> PGConnection
extern pg_prepare(conn: PGConnection, sql: string) -> PGStatement
extern pg_bind(stmt: PGStatement, params: [any]) -> PGStatement
extern pg_execute(stmt: PGStatement) -> PGResult

# Return type
type QueryResult:
    rows: [Row]
    affected: int
    columns: [string]
~

# Helper functions
func get_connection_from_env(var: string) -> PGConnection:
    conn_string = get_env(var)
    return pg_connect(conn_string)
~

# Handler - interprets $1, $2, etc. as positional parameters
func postgres_handler(call: KindCall) -> QueryResult:
    conn = get_connection_from_env("DATABASE_URL")

    # Use parameterized query for safety
    stmt = pg_prepare(conn, call.body)
    stmt = pg_bind(stmt, call.param_values)
    result = pg_execute(stmt)

    return QueryResult(rows: result.rows, affected: result.affected, columns: result.columns)
~

# Kind declaration (makes sqlquery available to importers)
kind sqlquery -> QueryResult via postgres_handler

```

Library documentation would specify:

- Use `$1`, `$2`, etc. for positional parameters
- Parameters are bound safely (no SQL injection)
- Set `DATABASE_URL` environment variable for connection

Import and Conflict Resolution

Basic Import

```
coex

import postgresql

sqlquery get_users() -> QueryResult:
    SELECT * FROM users

~
```

Multiple Implementations

When multiple libraries implement the same kind name:

```
coex

import postgresql
import mysql
```

This produces a compile error:

```
error: Conflicting kind declarations for 'sqlquery'
--> postgresql.coex:45: kind sqlquery -> QueryResult via postgres_handler
--> mysql.coex:42: kind sqlquery -> QueryResult via mysql_handler

= help: Use qualified imports to disambiguate:
    import postgresql as pg
    import mysql as my
```

Qualified Usage

```
coex
```

```
import postgresql as pg
import mysql as my

pg.sqlquery get_users() -> QueryResult:
    SELECT * FROM users # PostgreSQL syntax
~

my.sqlquery get_products() -> QueryResult:
    SELECT * FROM products # MySQL syntax
~
```

Aliased Import

```
coex

import postgresql as db # Use PostgreSQL as "db"

db.sqlquery get_users() -> QueryResult:
    SELECT * FROM users
~
```

Or import the kind directly:

```
coex

import postgresql.sqlquery # Just the kind, unqualified

sqlquery get_users() -> QueryResult:
    SELECT * FROM users
~
```

Body Capture

Termination

The body extends from the `(:)` to the closing `(~)`. The body is captured preserving:

- Whitespace and indentation
- Newlines
- All characters except the terminating `(~)`

No Processing

The compiler performs no processing on the body. It is captured as raw text and passed to the handler unchanged. Whatever placeholder syntax the library uses (`{!}`, `{{name}}`, `%name%`, `{name}`, etc.) is passed through literally.

No Nested Block Processing

The compiler does not look for nested structures. This body is captured as-is:

```
coex

jsontemplate config(appname: string) -> JSON:
  {
    "name": "{appname}",
    "nested": {
      "value": 42
    }
  }
~
```

The handler receives the exact text including all braces.

Implementation Requirements

Parser Changes

1. Add `kind` as a keyword
2. Parse kind declarations: `kind NAME -> TYPE via HANDLER`
3. Track declared kinds in scope
4. When encountering `KINDNAME funcname(params):`, switch to body capture mode
5. Capture all text until `~` as raw string
6. Generate desugared `func` that calls handler

Kind Registry

```
python
```

```
class KindRegistry:
    """Tracks declared function kinds."""

    kinds: Dict[str, KindDeclaration]

    def declare(self, name: str, return_type: Type, handler: str, module: str):
        if name in self.kinds:
            existing = self.kinds[name]
            if existing.module != module:
                raise ConflictError(f"Kind '{name}' already declared in {existing.module}")

        self.kinds[name] = KindDeclaration(
            name=name,
            return_type=return_type,
            handler=handler,
            module=module
        )

    def lookup(self, name: str) -> Optional[KindDeclaration]:
        return self.kinds.get(name)
```

Body Capture

```
python
```

```
class BodyCapture:
    """Capture raw body text for user-defined kinds."""

    def capture(self, lexer: Lexer) -> str:
        """Read until ~ appears alone or at statement position."""
        body_chars = []

        while True:
            char = lexer.read_char()

            if char == '~' and self._is_terminator_position(lexer):
                break

            if char == '$' and lexer.peek_char() == '$':
                # Escaped $, emit single $
                lexer.read_char() # consume second $
                body_chars.append('$')
            else:
                body_chars.append(char)

        return ''.join(body_chars).strip()
```

Parameter Substitution

None. The compiler passes the raw body to the handler. The handler is responsible for interpreting placeholders.

Code Generation

python

```
class KindCodeGenerator:
```

```
    """Generate desugared function for kind usage."""
```

```
def generate(self, kind_func: KindFunction, kind_decl: KindDeclaration) -> FuncDeclaration:
```

```
    # Build KindCall construction - body is passed as-is
```

```
    kind_call = f"KindCall("
```

```
        name: "{kind_func.name}",
```

```
        param_names: [{', '.join(f"{p.name}" for p in kind_func.params)}],
```

```
        param_values: [{', '.join(p.name for p in kind_func.params)}],
```

```
        body: {self._string_literal(kind_func.body)}
```

```
    )"
```

```
    # Build wrapper function
```

```
    return FuncDeclaration(
```

```
        name=kind_func.name,
```

```
        params=kind_func.params,
```

```
        return_type=kind_decl.return_type,
```

```
        body=f"return {kind_decl.handler}({kind_call})"
```

```
    )
```

```
def _string_literal(self, text: str) -> str:
```

```
    """Escape text for use as string literal."""
```

```
    escaped = text.replace("\\", '\\\\').replace("'", '\\\'').replace('\n', '\\n')
```

```
    return f'"{escaped}"'
```

Standard Library Considerations

Coex provides the standard `KindCall` type and helper utilities:

```
coex
```

```
# In stdlib/kind.coex
```

```
type KindCall:
```

```
  name: string
```

```
  param_names: [string]
```

```
  param_values: [any]
```

```
  body: string
```

```
~
```

```
# Get parameter value by name
```

```
func get_param(call: KindCall, name: string) -> any?:
```

```
  for i in range(len(call.param_names)):
```

```
    if call.param_names[i] == name:
```

```
      return call.param_values[i]
```

```
    ~
```

```
  ~
```

```
  return nil
```

```
~
```

```
# Helper for {{name}} style interpolation
```

```
func interpolate_double_curly(template: string, names: [string], values: [any]) -> string:
```

```
  result = template
```

```
  for i in range(len(names)):
```

```
    placeholder = "{{" + names[i] + "}"
```

```
    result = replace(result, placeholder, to_string(values[i]))
```

```
  ~
```

```
  return result
```

```
~
```

```
# Helper for {name} style interpolation
```

```
func interpolate_curly(template: string, names: [string], values: [any]) -> string:
```

```
  result = template
```

```
  for i in range(len(names)):
```

```
    placeholder = "{" + names[i] + "}"
```

```
    result = replace(result, placeholder, to_string(values[i]))
```

```
  ~
```

```
  return result
```

```
~
```

```
# Helper for %name% style interpolation
```

```
func interpolate_percent(template: string, names: [string], values: [any]) -> string:
```

```
  result = template
```

```
  for i in range(len(names)):
```

```
placeholder = "%" + names[i] + "%"
result = replace(result, placeholder, to_string(values[i]))
~
return result
~
```

Library authors can use these helpers or implement their own interpolation logic.

Examples

SQL (Positional Parameters)

```
coex

# User code
import postgresql

# Library convention: $1, $2, etc.
sqlquery find_user(id: int) -> QueryResult:
    SELECT * FROM users WHERE id = $1
~

sqlquery find_by_name_and_dept(name: string, dept: string) -> QueryResult:
    SELECT * FROM users WHERE name = $1 AND department = $2
~

func main():
    result = find_user(42)
    print(result.rows[0].name)
~
```

GraphQL (Named Placeholders)

```
coex
```

```
# graphql_client.coex
kind graphql -> GraphQLResult via graphql_handler

func graphql_handler(call: KindCall) -> GraphQLResult:
    endpoint = get_env("GRAPHQL_ENDPOINT")
    token = get_env("GRAPHQL_TOKEN")

    # Convention: {{name}} placeholders
    query = call.body
    for i in range(len(call.param_names)):
        query = replace(query, "{{" + call.param_names[i] + "}}",
                        to_string(call.param_values[i]))

    ~

    return http_post_json(endpoint, {query: query}, auth: token)

~
```

```
coex

# User code
import graphql_client

# Library convention: {{name}} placeholders
graphql get_user_posts(user_id: string) -> GraphQLResult:
    query {
        user(id: "{{user_id}}") {
            name
            posts {
                title
                content
            }
        }
    }

~
```

Shell Commands (Percent Placeholders)

```
coex
```

```
# shell.coex
```

```
kind shell -> ShellResult via shell_handler
```

```
type ShellResult:
```

```
    exit_code: int
```

```
    stdout: string
```

```
    stderr: string
```

```
~
```

```
func shell_handler(call: KindCall) -> ShellResult:
```

```
    command = call.body
```

```
    # Convention: %name% with shell escaping
```

```
    for i in range(len(call.param_names)):
```

```
        placeholder = "%" + call.param_names[i] + "%"
```

```
        escaped = shell_escape(to_string(call.param_values[i]))
```

```
        command = replace(command, placeholder, escaped)
```

```
~
```

```
    return exec_command(command)
```

```
~
```

```
coex
```

```
# User code
```

```
import shell
```

```
# Library convention: %name% placeholders
```

```
shell list_files(dir: string, pattern: string) -> ShellResult:
```

```
    find %dir% -name "%pattern%" -type f
```

```
~
```

```
func main():
```

```
    result = list_files("/home/user", "*.txt")
```

```
    print(result.stdout)
```

```
~
```

LLM Prompts (Curly Brace Placeholders)

```
coex
```

```
# anthropic.coex
```

```
kind llm -> LLMResponse via claude_handler
```

```
func claude_handler(call: KindCall) -> LLMResponse:
```

```
    api_key = get_env("ANTHROPIC_API_KEY")
```

```
    prompt = call.body
```

```
    # Convention: {name} placeholders
```

```
    for i in range(len(call.param_names)):
```

```
        prompt = replace(prompt, "{" + call.param_names[i] + "}",  
                           to_string(call.param_values[i]))
```

```
    ~
```

```
    return anthropic_messages_api(
```

```
        api_key: api_key,
```

```
        model: "claude-sonnet-4-20250514",
```

```
        messages: [{role: "user", content: prompt}]
```

```
    )
```

```
    ~
```

```
coex
```

```

# User code
import anthropic

# Library convention: {name} placeholders
llm summarize(text: string, max_bullets: int) -> LLMResponse:
    Summarize the following text in at most {max_bullets} bullet points:

    {text}
~

llm translate(text: string, target_lang: string) -> LLMResponse:
    Translate the following to {target_lang}:

    {text}
~

func main():
    article = load_file("article.txt")
    summary = summarize(article, 5)
    print(summary.content)
~

```

Configuration Templates (Double Curly Braces)

```

coex

# kubernetes.coex
kind k8s -> DeployResult via k8s_handler

func k8s_handler(call: KindCall) -> DeployResult:
    manifest = call.body

    # Convention: {{name}} placeholders
    for i in range(len(call.param_names)):
        manifest = replace(manifest, "{{" + call.param_names[i] + "}}",
                           to_string(call.param_values[i]))
    ~

    return kubectl_apply(manifest)
~

```

```

coex

```

```
# User code
import kubernetes

# Library convention: {{name}} placeholders
k8s deploy_service(name: string, image: string, replicas: int) -> DeployResult:
    apiVersion: apps/v1
    kind: Deployment
    metadata:
        name: {{name}}
    spec:
        replicas: {{replicas}}
        selector:
            matchLabels:
                app: {{name}}
    template:
        metadata:
            labels:
                app: {{name}}
        spec:
            containers:
                - name: {{name}}
                  image: {{image}}
                  ports:
                    - containerPort: 8080

~

func main():
    result = deploy_service("myapp", "myregistry/myapp:latest", 3)
    print("Deployed: ", result.status)

~
```

Testing

Parser Tests

```
python
```

```

def test_parse_kind_declaration():
    source = "kind sqlquery -> QueryResult via sql_handler"
    ast = parse(source)

    assert ast.kinds[0].name == "sqlquery"
    assert ast.kinds[0].return_type == "QueryResult"
    assert ast.kinds[0].handler == "sql_handler"

def test_parse_kind_function():
    source = """
kind sqlquery -> QueryResult via handler

sqlquery get_users(min_age: int) -> QueryResult:
    SELECT * FROM users WHERE age > $1

~
"""
    ast = parse(source)

    func = ast.functions[0]
    assert func.kind_name == "sqlquery"
    assert func.params[0].name == "min_age"
    assert "$1" in func.body # Body captured as-is

```

Desugaring Tests

```
python
```

```

def test_desugar_kind_function():
    source = """
kind greet -> string via greet_handler

greet say_hello(name: string) -> string:
    Hello, {{name}}! Welcome.

~
"""

    desugared = desugar(parse(source))

    # Should produce a regular func
    func = desugared.functions[0]
    assert func.kind == FunctionKind.FUNC
    assert "greet_handler" in func.body
    assert "KindCall" in func.body

def test_body_passed_raw():
    source = """
kind greet -> string via greet_handler

greet say_hello(name: string) -> string:
    Hello, {{name}}! Welcome.

~
"""

    desugared = desugar(parse(source))
    # Body should contain literal {{name}}, not substituted
    assert "{{name}}" in desugared.functions[0].body

```

Integration Tests

python

```

def test_kind_end_to_end():
    source = """
type KindCall:
    name: string
    param_names: [string]
    param_values: [any]
    body: string
~

func echo_handler(call: KindCall) -> string:
    return call.body
~

kind echo -> string via echo_handler

echo say_something(msg: string) -> string:
    Message: {msg}
~

func main():
    result = say_something("hello")
    print(result)
~
"""

# Handler returns raw body - no substitution
assert run(source) == "Message: {msg}"

def test_handler_does_substitution():
    source = """
type KindCall:
    name: string
    param_names: [string]
    param_values: [any]
    body: string
~

func template_handler(call: KindCall) -> string:
    result = call.body
    for i in range(len(call.param_names)):
        placeholder = "{" + call.param_names[i] + "}"
        result = replace(result, placeholder, to_string(call.param_values[i]))
~

```

```
return result
```

```
~
```

```
kind template -> string via template_handler
```

```
template greet(name: string, age: int) -> string:
```

```
    Hello {name}, you are {age} years old.
```

```
~
```

```
func main():
```

```
    print(greet("Alice", 30))
```

```
~
```

```
"""
```

```
assert run(source) == "Hello Alice, you are 30 years old."
```

Diagnostics

Compile-Time Errors

```
error: Unknown function kind 'sqlquery'
```

```
--> source.coex:5:1
```

```
|
```

```
5 | sqlquery get_users() -> QueryResult:
```

```
| ^^^^^^^^^
```

```
|
```

```
= help: Did you forget to import a library that declares this kind?
```

```
= help: Available kinds: formula, func, task, thread, declare
```

```
error: Kind handler 'sql_handler' not found
```

```
--> source.coex:3:35
```

```
|
```

```
3 | kind sqlquery -> QueryResult via sql_handler
```

```
| ^^^^^^^^^^^^^^^
```

```
|
```

```
= note: The handler must be a func visible at this point
```

```
error: Kind handler has wrong signature
--> source.coex:3:35
|
3 | kind sqlquery -> QueryResult via wrong_handler
|               ^^^^^^^^^^^^^^^^^
|
|
= note: Expected: func(KindCall) -> QueryResult
= note: Found: func(string) -> string
```

Summary

User-defined function kinds provide:

1. **Lightweight DSL syntax:** DSLs look like native Coex functions
2. **Library-driven:** No compiler changes for new DSLs
3. **Swappable backends:** Same kind name, different implementations
4. **Handler controls everything:** Compiler just packages and passes through
5. **Environment configuration:** Credentials never in code

Implementation requires:

- `kind` keyword and declaration parsing
- Body capture mode in lexer
- Kind registry for tracking declarations
- Desugaring pass to convert kind functions to handler calls
- Standard `KindCall` type in stdlib

The compiler's job is minimal: capture the body as raw text, package it with parameter names and values, and call the handler. The handler does all interpretation, substitution, escaping, and execution. Each library documents its own placeholder convention (`$1`, `{{name}}`, `%name%`, `{name}`, etc.).